Modbus RS485 Troubleshooting Quick Reference

Emile Ackbarali

June 20, 2015

Modbus RS485 Troubleshooting Quick Reference

Emile Ackbarali

About The Author

Hi, I'm Emile Ackbarali. I am an Automation Engineer and ASP.NET Web Developer. This guide is intended for Automation engineers and technicians who do the practical implementation and troubleshooting of Modbus RS485 networks in the field. It is meant to be a means for rapidly retrieving key pieces of data so that network issues can be resolved in the fastest possible manner.

The information in here was derived from information I personally keep with me when I am performing my implementation and troubleshooting of Modbus networks in the field. I hope that it serves you as well as it has served me over the years.

Modbus RS485 Troubleshooting Quick Reference

Emile Ackbarali

Table of Contents

Chapter 1 - RS485 Troubleshooting Techniques 3

Chapter 2 - Modbus Reference Information 6

Chapter 3 - Modbus Troubleshooting Techniques 11

Summary 14

Resources 15

Modbus RS485 Troubleshooting Quick Reference

Emile Ackbarali

Chapter 1 - RS485 Troubleshooting Techniques

A breakdown in communication on a Modbus RS485 network can be for several reasons. In this chapter I am going to concentrate on the causes of communication breakdown from the perspective of RS485.

The problems caused by RS485 based on my experience fall into two areas:

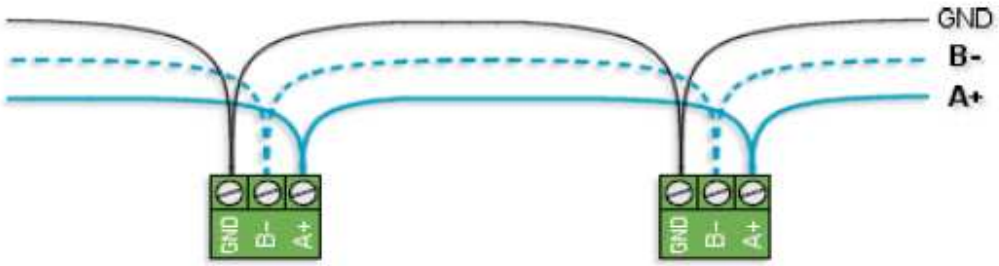
(1) Problems with physical wire connections between the devices.

(2) Mismatched communication settings between devices on the network.

(1) Problems with the physical wire connections between devices.

All devices that communicate via RS485 must obviously have an RS485 port. This port may take the form of simple screw terminals or it may be implemented via a DB9 or RJ45 connector. However it is built, in the majority of cases you will be looking for just three pinouts. They are +, - and ground. Some devices will have the pinouts labeled as A, B and ground. And you may, though not often, also see these same pinouts labeled as Tx/Rx+, Tx/Rx- and ground.

The diagram below illustrates the typical way in which RS485 devices are connected using a daisy chain method.



If you have connected your network in this way and you are getting no communication or unreliable communication, here are the checks that you should do:

(1) Start at the very basic level and check to see that all of your connections have been properly terminated and are tightly connected. As simple as it sounds, loose wire connections can cause intermittent communication issues on a RS485 network, which can be a task to troubleshoot.

(2) One of the devices may have a bad RS485 port. Try replacing the RS485 devices one at a time with a known good device. If you're doing multi-drop RS485 then any one device with a bad serial port can cause problems for all the other devices on that cable.

(3) Even though RS485 is very resistant to electrical noise, if the communications cable is run too close to machines / equipment that generate significant electrical noise, then this could cause an issue. If this is the case, the cables should be re-run so that it is not as exposed.

(4) A ground loop. If more than one device on the RS485 cable connects the shield to ground it can cause a ground loop. A ground loop can interfere with data by distorting the RS485 signal. If the shield is grounded in the middle of the cable (because it's accidentally touching some grounded metal object for instance) it can also cause a ground loop. The shield of your communications cable should only be grounded on one end.

(5) Termination and/or biasing problems. This is usually only a problem if the cable is fairly long or if there are lots of RS485 devices on the same cable. There have been very few instances in my experience where I had to explicitly use terminating resistors on an RS485 network, but I will still discuss this a bit here.

Termination and Biasing

Termination matches impedance of a node (transmitter/receiver) to the impedance of the transmission line (cable). When impedances are mismatched, the transmitted signal is not completely absorbed by the load and a portion reflects back into the line.

Termination, however, increases load on the drivers, increases installation complexity, changes biasing requirements, and makes system modification more difficult.

Also, adding termination greatly increases power consumption and requires the network to be re-biased. Termination may not have any benefit when used at low data rates or on short runs. Termination is not recommended when using port-powered RS-232 to RS-485 converters.

If you use terminators, understand their pros and cons. You'll need resistors of more than 90O. Typically, they'll be placed only at the extreme ends of the data line. Place no

more than two terminations in any system that does not use repeaters. Always recalculate the bias resistors when you add termination to a system.

(2) Mismatched communication settings between devices on the network.

If all of your cabling is looking good, then the next thing to look at is the configuration of the communication settings on all of the devices on the network. The communication settings will essentially be the RS485 port settings on every device. Each setting will consist of the following data parameters:

Baud rate, Start bits, Parity, Stop Bits.

These settings can have many different combinations, which is fine. What you have to check and ensure is that the settings are exactly the same from device to device.

Mismatched port settings most times will result in no communication taking place on the network. A master would send a query to a slave and the slave would simply not respond.

Or even worse, the slave may respond! Let me explain this one. I have had instances of a master sending a query to a slave and even though the slave had a different port setting, it would send back a response with a Modbus error code. Due to the mismatched port settings, the slave would interpret the data sent in some way and "think" it was an erroneous Modbus query.

So the bottom line is to check to ensure all the devices on your network have the same RS485 port settings.

Modbus RS485 Troubleshooting Quick Reference

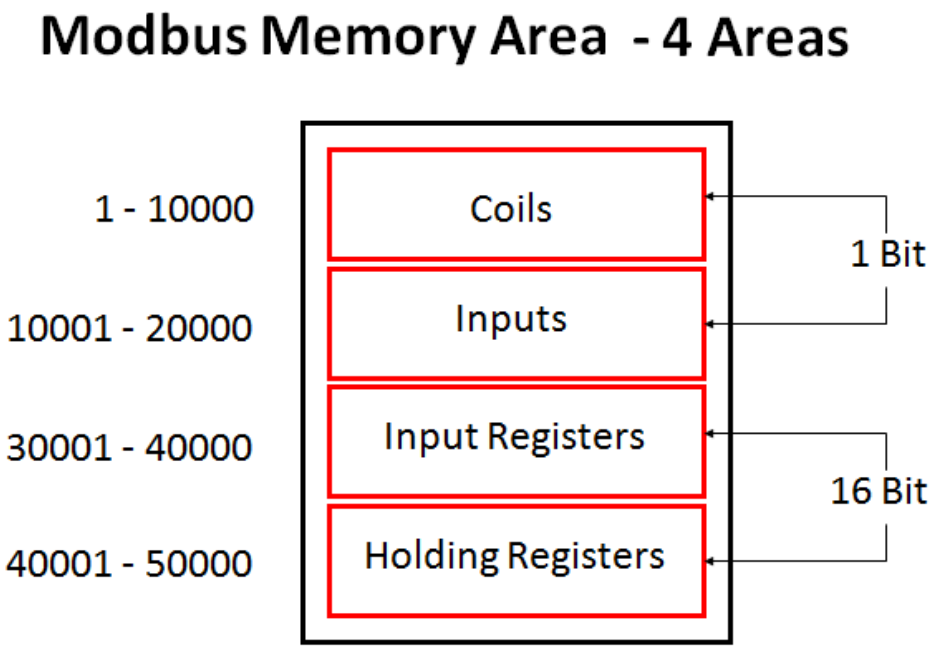
Emile Ackbarali

Chapter 2 - Modbus Reference Information

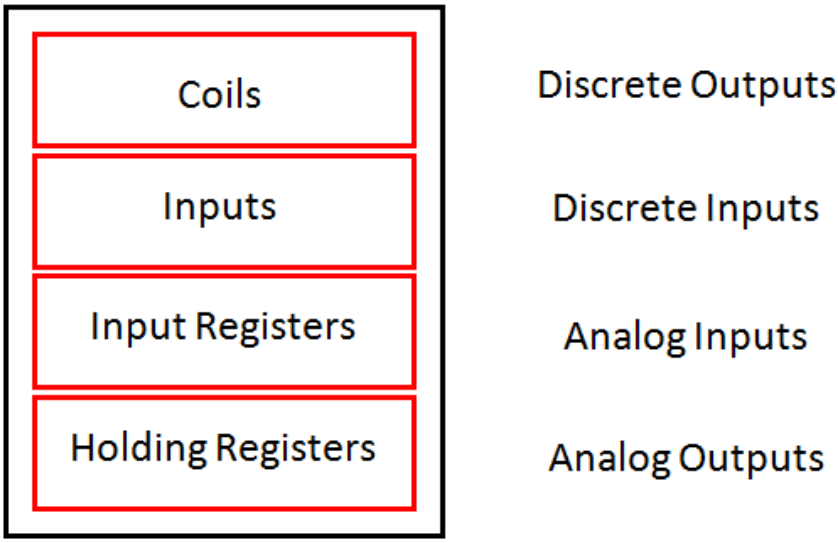
This chapter contains the key Modbus information that you will need to quickly refer to in the field. I may refer to this information when I talk about troubleshooting steps in Chapter 3 of this book.

(1) The Modbus Memory Map

The following 2 diagrams illustrate the typical Modbus memory areas.



Modbus Memory Area - 4 Areas



(2) Most Common Modbus Function Codes

01 - Read Coil Status

02 - Read Input Status

03 - Read Holding Registers

04 - Read Input Registers

05 - Force Single Coil

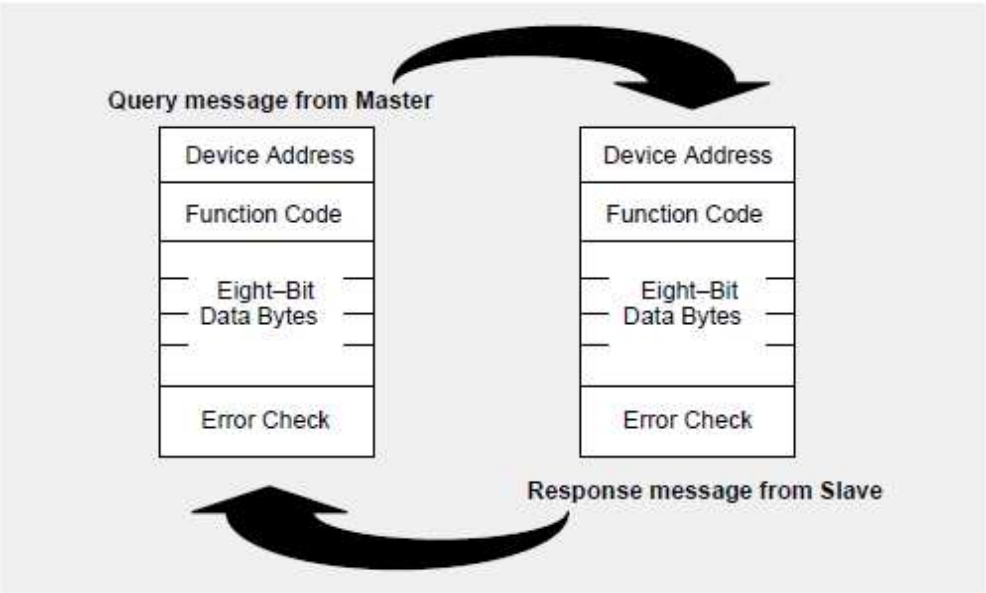
06 - Preset Single Register

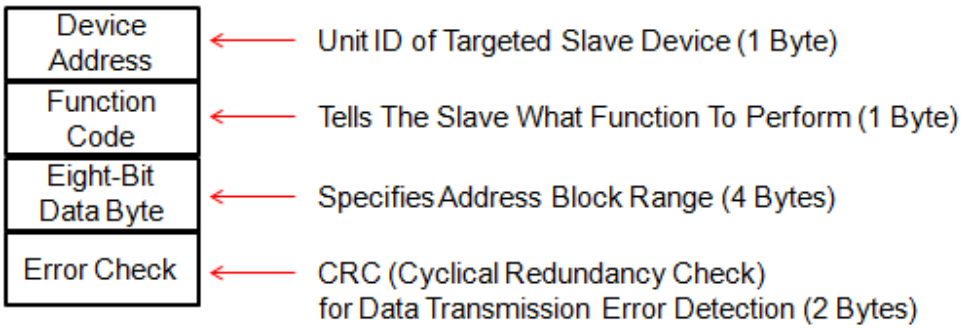
15 - Force Multiple Coils

16 - Preset Multiple Registers

(3) Modbus Typical Query-Response Cycle

The following diagrams illustrate the makeup of the data byte streams that make up a typical Modbus Query-Response cycle.





(4) Modbus Exception Codes

01 - ILLEGAL FUNCTION: The function code received in the query is not an allowable action for the slave. If a Poll Program Complete command was issued, this code indicates that no program function preceded it.

02 - ILLEGAL DATA ADDRESS: The data address received in the query is not an allowable address for the slave.

03 - ILLEGAL DATA VALUE: A value contained in the query data field is not an allowable value for the slave.

04 - SLAVE DEVICE FAILURE: An unrecoverable error occurred while the slave was attempting to perform the requested action.

05 - ACKNOWLEDGE: The slave has accepted the request and is processing it, but a long duration of time will be required to do so. This response is returned to prevent a timeout error from occurring in the master. The master can next issue a Poll Program Complete message to determine if processing is completed.

06 - SLAVE DEVICE BUSY: The slave is engaged in processing a long–duration program command. The master should retransmit the message later when the slave is free.

07 - NEGATIVE ACKNOWLEDGE: The slave cannot perform the program function received in the query. This code is returned for an unsuccessful programming request using function code 13 or 14 decimal. The master should request diagnostic or error information from the slave.

08 - MEMORY PARITY ERROR: The slave attempted to read extended memory, but detected a parity error in the memory. The master can retry the request, but service may be required on the slave device.

(5) Example of an Exception Code Query-Response

QUERY

Byte	Contents	Example
1	Slave Address	0A
2	Function	01
3	Starting Address Hi	04
4	Starting Address Lo	A1
5	No. of Coils Hi	00
6	No. of Coils Lo	01
7	LRC	4F

EXCEPTION RESPONSE

Byte	Contents	Example
1	Slave Address	0A
2	Function	81
3	Exception Code	02
4	LRC	73

In the example above, the master addresses a query to slave device 10 (0A hex). The function code (01) is for a Read Coil Status operation. It requests the status of the coil at address 1245 (04A1 hex). Note that only that one coil is to be read, as specified by the number of coils field (0001).

If the coil address is non–existent in the slave device, the slave will return the exception response with the exception code shown (02). This specifies an illegal data address for the slave.

Modbus RS485 Troubleshooting Quick Reference

Emile Ackbarali

Chapter 3 - Modbus Troubleshooting Techniques

In this chapter, I am assuming that you would have already made all of the checks to the cabling and that you also ensured that the RS485 port settings were the same for all of the devices. Hence I will be concentrating on troubleshooting techniques directly related to Modbus protocol issues. Below are the methods that I use to implement new and troubleshoot existing Modbus networks.

Let's assume that you encounter an existing Modbus RS485 network or you have just set up one and no communication is taking place at all on the network. Here are the checks you should make:

(1) Unit ID Check

Every device on a Modbus network must have a unique Unit ID. If a network is experiencing issues, the first thing to check is that the Unit ID on each device is set up such that there are no duplicates. Even though you may have checked this already, check it again. The Unit ID of some devices are set using dip switches, or a local display. Some however require set up via connection to a computer with use of the appropriate manufacturer's software.

(2) Modbus Master Configuration Check

You must remember that the Modbus master is responsible for orchestrating all of the communication that occurs on the network. The Modbus master must therefore be configured to know the following:

(i) The Unit ID of each slave on the network.

(ii) What registers to read from and write to for each respective slave.

This information is user configured, in most cases via the use of a computer and the appropriate manufacturer's software.

You must ensure that the Modbus master has the correct list of slave Unit ID's on the network as well as the correct register listing for each respective slave.

(3) Modbus Master Substitution

In order to execute this method, you need to have a computer (preferably a laptop) and a usb-to-RS485 converter connected to it. Then you need to run a Modbus master application on your computer so that it can send Modbus query messages onto the network. My preference for a Modbus master application is Modscan32 from win-tech. You can, however, use one that you are comfortable with.

The idea here is to remove the Modbus master device and then connect the usb-to-RS485 converter to the network and then use the laptop as the Modbus master to try to communicate with each slave on the network. If you are able to communicate effectively, then you would know that the problem exists in the original Modbus master device that you would have disconnected. At that point you can re-look at the original Modbus master configuration.

If it is the Modscan32 software can't communicate with any of the slave devices on the network, then what you should do is to disconnect all of the slave devices and then re-connect them one at a time and try to communicate with each individually, in the event that one of the slave devices may be causing a problem on the network.

In the event that you are getting communication to the slaves but no data, you should check the Response data streams to see if there are exception responses coming back. If there are, you can then look at the exception codes being returned.

(4) Modbus Slave Substitution

Typically I would use this method when I am dealing with a new piece of equipment that is to act as a Modbus master. I use this method to verify that the configuration method indicated in the user manual for the Modbus master is accurate.

So here you would use your laptop and usb-to-RS485 converter as well. The idea here is to run a Modbus slave simulator application on the laptop so that it acts as a Modbus slave. There are several applications you can run but I prefer to use Modsim32 from win-tech.

So the Modbus master is connected to the RS485 converter and therefore to the laptop. The Modsim32 application is then run and a simulated slave is created within it. The Modbus master device will then be configured to read data from the simulated slave. The data values in Modsim32 can then be changed and the Modbus master checked to see if it is reading the data values properly.

So using Modscan32 and Modsim32, you can controlled test situations that allow for effective troubleshooting of the network.

Modbus RS485 Troubleshooting Quick Reference

Emile Ackbarali

Summary

The information and techniques that I have put down in this guide are the very same that I use when I am out in the field implementing and troubleshooting Modbus RS485 networks. I have tried to put down the techniques in the order in which I use them so as to solve issues in the least possible time.

If you have any questions / comments based on what I have written here, I would be happy to hear them. Please contact me on ea.innovate@gmail.com.

Modbus RS485 Troubleshooting Quick Reference

Emile Ackbarali

Resources

You can download Modscan32 and Modsim32 at www.win-tech.com.

The usb-to-RS485 converter that I use is called the " Baitaihem USB to RS485 USB-485 DB9 Serial Port COM". I bought it on www.amazon.com for just under $20.00 inclusive of shipping. I have used it for quite a while now with no complaints. You can simply type in the name in Amazon and you will find it.

A picture of it is shown below: